Università di Pisa
Advanced Software Engineering — Laboratorio · A.A. 2026/27

Monster Tamers

Architettura del backend e decisioni di progetto

Documento di progetto

Gruppo	Nome del gruppo e membri da inserire
Versione	1.0
Data	15 ottobre 2026
Stato	Decisioni approvate dal gruppo; parametri di gioco da bilanciare
Docente	Alessandro Bocci

Sommario

Questo documento descrive l'architettura del backend di *Monster Tamers*, il gioco di collezione di mostri realizzato come progetto del laboratorio di Advanced Software Engineering, e motiva le decisioni prese dal gruppo.

Il backend è un'applicazione distribuita composta da **sei servizi Quarkus** (`auth`, `players`, `monsters`, `teams`, `encounters`, `history`), ciascuno con il proprio database PostgreSQL, esposti attraverso un unico **gateway Nginx**. I servizi comunicano solo tramite **REST sincrono**; l'autenticazione usa **JWT** firmati dal servizio `auth` e verificati localmente da tutti gli altri.

Il gioco è strutturato come un **roguelike a run**: il giocatore sceglie tre mostri iniziali, affronta encounter interattivi a turni in otto stage di difficoltà crescente e vince la run battendo il boss finale. Una sconfitta azzera la run; account, storico e run passate restano consultabili.

Il documento copre, nell'ordine: requisiti e vincoli (Sezione 2), regole di gioco (Sezione 3), architettura generale (Sezione 4), dettaglio dei servizi con API e dati (Sezione 5), flussi principali (Sezione 6), consistenza dei dati (Sezione 7), sicurezza (Sezione 8), deployment e test (Sezione 9), registro delle decisioni con le alternative scartate (Sezione 10) e punti aperti (Sezione 11).

Indice

1 Introduzione **4**

1.1 Scopo del documento 4
1.2 Contesto 4
1.3 Documenti di riferimento 4
1.4 Convenzioni 4
1.5 Glossario 5

2 Requisiti e vincoli **5**

2.1 Requisiti tecnici 5
2.2 User stories 5
2.3 Vincoli di progetto 6

3 Il gioco **6**

3.1 Visione d'insieme 6
3.2 Giocatore 7
3.3 Mostri 7
3.4 Collezione e istanze 7
3.5 Starter 8
3.6 Team 8
3.7 Encounter e stage 8
3.8 Battaglia 9

3.9 Run: fine, reset e resa . . . 11

4 Architettura **11**

4.1 Stile architetturale . . . 11
4.2 Vista dei componenti . . . 11
4.3 Responsabilità dei servizi . . . 12
4.4 Gateway . . . 12
4.5 Comunicazione tra servizi . . . 12
4.6 Autenticazione e autorizzazione . . . 13
4.7 Stack tecnologico . . . 14

5 Servizi **14**

5.1 `auth` . . . 14
5.2 `players` . . . 15
5.3 `monsters` . . . 16
5.4 `teams` . . . 17
5.5 `encounters` . . . 18
5.6 `history` . . . 19

6 Flussi principali **20**

6.1 Registrazione e scelta degli starter . . . 20
6.2 Avvio e svolgimento di una battaglia . . . 21
6.3 Fine della battaglia e reset della run . . . 22

7 Dati e consistenza **22**

7.1 Un database per servizio . . . 22
7.2 Consistenza tra servizi . . . 22
7.3 Migrazioni e seed . . . 23
7.4 Immagini . . . 23

8 Sicurezza **23**

9 Deployment e test **24**

9.1 Deployment . . . 24
9.2 Strategia di test . . . 25

10 Registro delle decisioni **25**

10.1 Architettura . . . 25
10.2 Regole di gioco . . . 26

10.3 Modifiche alle decisioni precedenti 27

11 Limiti noti e punti aperti **28**

11.1 Limiti noti 28

11.2 Punti aperti 28

A Tracciabilità delle user stories **28**

B Catalogo delle abilità **29**

1 Introduzione

1.1 Scopo del documento

Il documento ha tre obiettivi:

1. descrivere **cosa** fa il backend: componenti, responsabilità, API e dati;
2. spiegare **perché** è fatto così, riportando per ogni decisione le alternative considerate e le conseguenze;
3. servire da riferimento comune al gruppo durante lo sviluppo e da base per la consegna dell'architettura, per il report finale e per la discussione orale.

1.2 Contesto

Monster Tamers è un gioco di collezione di mostri. Il ciclo di gioco minimo richiesto dal docente è *Prepara* → *Affronta* → *Esito* → *Progresso*: il giocatore raccoglie mostri, forma un team, affronta un *encounter* contro un mostro selvatico e, in base all'esito, fa progressi e ottiene nuovi mostri.

Il focus del progetto è il backend, che deve gestire collezione globale e collezioni personali, selezione del team, encounter e storico persistente, il tutto tramite un'interfaccia RESTful. Il frontend è minimale e non contiene logica di business.

1.3 Documenti di riferimento

Documento	Contenuto
`Project Kickoff.pdf`	Presentazione del progetto (07/10/2026)
`Monster_Tamers_Backlog.xlsx`	28 user stories con priorità
`Monster_Tamers_Riepilogo.pdf`	Riepilogo in italiano di consegna e requisiti
`area_giocatori_mostri.md`	Decisioni su giocatori e mostri (D1–D13, G1–G9) e acceptance criteria
`architettura_backend.md`	Versione sintetica di questo documento
`backend/monsters/seed/`	Catalogo dei mostri, abilità ed elementi (JSON)

1.4 Convenzioni

- Le decisioni sono identificate da un prefisso: **A** per l'architettura, **R** per le regole di gioco. Le decisioni **D** (dati) e **G** (catalogo) sono quelle del documento dell'area giocatori e mostri.
- I valori marcati come *iniziali* (formule, soglie, livelli) sono parametri di bilanciamento: la struttura è decisa, i numeri possono cambiare dopo le simulazioni.
- Gli endpoint pubblici hanno il prefisso `/api` e passano dal gateway; quelli con prefisso `/internal` sono raggiungibili solo tra servizi.
- Nomi di servizi, tabelle e campi sono in inglese, come nel codice; il testo è in italiano.

1.5 Glossario

Termine	Significato
Specie	Un mostro del catalogo globale (100 specie), identificato da `monster_id`.
Istanza	Un mostro posseduto da un giocatore, identificato da `owned_id`, con livello e XP propri. Della stessa specie possono esistere più istanze.
Run	Una partita completa: dalla scelta degli starter alla vittoria finale, alla sconfitta o alla resa.
Stage	Livello di difficoltà della run, da 1 a 8.
Template	Definizione statica di un encounter: stage, elemento, livelli e rarità dei selvatici possibili.
Selvatico	Il mostro avversario di un encounter, estratto dal catalogo all'avvio.
Boss	Encounter speciale di fine stage con un selvatico prefissato e più forte.
Battaglia	Esecuzione concreta di un encounter: stato in corso, turni, esito.
Staffetta	Formato in cui i mostri del team combattono uno alla volta, nell'ordine del team.

2 Requisiti e vincoli

2.1 Requisiti tecnici

Requisito	Come viene soddisfatto	Sezione
Applicazione distribuita	Sei servizi indipendenti in container separati, ognuno con il proprio database	4
Interfaccia REST	Tutte le operazioni del giocatore sono endpoint REST documentati con OpenAPI	5
Uno o più database	Un'istanza PostgreSQL per servizio; storico degli encounter persistente	7
Meccanismi di sicurezza	HTTPS, JWT, bcrypt, autorizzazione per risorsa, endpoint interni protetti	8
Test unitari e di performance	JUnit, test di API con RestAssured, Locust sul gateway	9
Analisi di sicurezza	Misure elencate per minaccia; analisi completa per la consegna finale	8
Nessuna logica nel frontend	Regole, calcoli ed estrazioni casuali solo nel backend	3

2.2 User stories

Il backlog contiene 28 user stories: 18 per il primo prototipo, 2 aggiuntive per la consegna finale (US-05 e US-25, entrambe sulla sicurezza) e 8 opzionali. Le scelte di questo documento coprono anche alcune opzionali senza costi aggiuntivi rilevanti: US-04 (nome e avatar), US-07 (scelta del primo mostro, tramite gli starter), US-13 (sviluppo dei mostri, tramite livelli e XP), US-21 (fuga) e US-22 (scelta delle azioni, tramite la battaglia interattiva). Non sono coperte US-08 (fazioni),

US-26 e US-27 (amici e sfide). La corrispondenza completa tra user stories, servizi ed endpoint è in Appendice A.

2.3 Vincoli di progetto

- Gruppo di 3–4 persone; ogni membro deve saper discutere individualmente la propria parte all'orale.
- Consegna finale il 15 dicembre; prototipo opzionale il 10 novembre con le 18 user stories della prima consegna.
- Principio KISS richiesto dal docente: la complessità va aggiunta solo dove porta un beneficio misurabile.
- Framework scelto dal gruppo: Quarkus (Java).

3 Il gioco

3.1 Visione d'insieme

Il gioco è un **roguelike a run** (R1). Una run inizia con la scelta di tre mostri iniziali e si sviluppa in **otto stage** di difficoltà crescente (R2). In ogni stage il giocatore affronta encounter normali, che può ripetere, e un boss. Battere il boss dello stage 8 vince la run; perdere un encounter la fa terminare con un reset completo (R14).

Il ciclo di gioco richiesto dal docente si ritrova così:

Fase	In Monster Tamers
Prepara	Scelta degli starter, consultazione della collezione, creazione e modifica dei team
Affronta	Battaglia interattiva a turni contro un mostro selvatico
Esito	Vittoria (il selvatico entra in collezione, XP al team), sconfitta (fine della run) o fuga
Progresso	Storico degli encounter, livelli dei mostri, avanzamento di stage, run passate

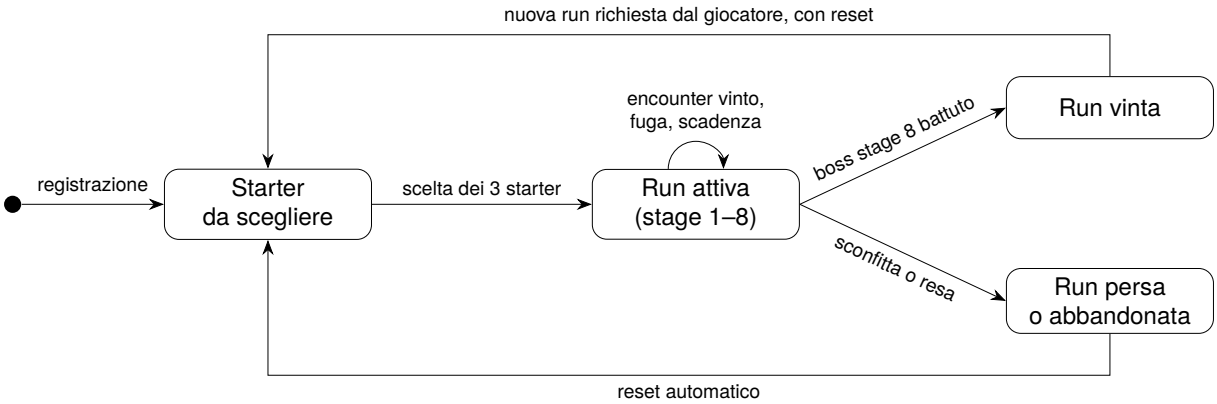


Figura 1. Ciclo di vita di una run dal punto di vista del giocatore.

3.2 Giocatore

Account e profilo sono separati (D1). L'**account** contiene solo le credenziali (username e hash della password) e vive nel servizio `auth`. Il **profilo** contiene i dati di gioco: nome visualizzato, avatar, run corrente e pool degli starter, e vive nel servizio `players`. I due condividono lo stesso identificativo `player_id`.

I progressi non sono duplicati in un unico campo: si ricavano dalla collezione (`monsters`), dalla run corrente (`encounters`) e dallo storico (`history`).

3.3 Mostri

Il catalogo contiene **100 specie** (D9), prodotte dalla pipeline in `tools/monster-gen/`. Ogni specie ha:

- un'**immagine** pixel art 64×64 servita come file statico (D8);
- un **elemento** fra sette elementi concettuali;
- una **rarità**: `common`, `rare` o `legendary`;
- quattro **statistiche** base intere 1–100 (`hp`, `attack`, `defense`, `speed`) con un tasso di **crescita** per livello (D5, D13);
- una **abilità** speciale scelta da un catalogo condiviso di 28 abilità (D12).

Elemento	Common	Rare	Legendary	Totale
Silence	10	4	1	15
Echo	10	4	1	15
Memory	10	5	1	16
Time	9	4	2	15
Chaos	1	4	3	8
Gravity	10	4	1	15
Dream	10	5	1	16
Totale	60	30	10	100

Tabella 1. Distribuzione del catalogo per elemento e rarità. La somma delle statistiche base cresce con la rarità: 148–174 per i common, 206–233 per i rare, 270–288 per i legendary.

Gli elementi formano un **ciclo** (Figura 2): ogni elemento è forte contro il successivo, debole contro il precedente e neutro con gli altri quattro. Chaos è un elemento speciale con pochi mostri, quasi tutti rari o leggendari.

Le **abilità** sono descritte da tre campi: tipo di effetto (`effect_type`), intensità (`effect_value`) e condizione di attivazione (`trigger`). La logica che le applica è solo nel backend. Il catalogo completo è in Appendice B.

3.4 Collezione e istanze

Ogni mostro posseduto è un'**istanza** con il proprio `owned_id`, livello ed esperienza; più istanze della stessa specie sono ammesse (D6). Ogni istanza appartiene a una run (`run_id`) e ha uno stato: `active` se appartiene alla run corrente, `archived` se appartiene a una run conclusa (R15). La collezione mostrata al giocatore è quella della run corrente; le collezioni delle run passate restano consultabili.

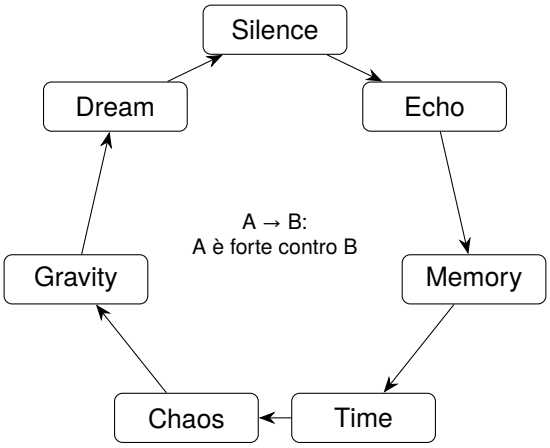


Figura 2. Ciclo degli elementi (`elements.json`). Un attacco forte ha moltiplicatore 1,5, uno debole 0,67.

Le statistiche effettive dipendono dal livello e sono calcolate solo dal servizio `monsters`:

$$s_{\text{eff}} = s_{\text{base}} + g_s \cdot (L - 1)$$

dove s_{base} è la statistica base della specie, g_s il suo tasso di crescita e L il livello dell'istanza (massimo 50).

3.5 Starter

Alla creazione dell'account, e all'inizio di ogni nuova run, il backend estrae a caso un **pool di 10 specie common** e lo salva nel profilo, così ricaricare la pagina non cambia l'offerta. Il giocatore sceglie **3 specie distinte** dal pool; le tre istanze entrano in collezione a livello 1 e la run si apre allo stage 1. La scelta si fa una volta per run.

3.6 Team

Prima di un encounter il giocatore prepara un team con le istanze della propria collezione. Le regole sono volutamente poche (R8):

- da **1 a 3** membri, numero che coincide con gli starter e permette di avere subito un team completo;
- ogni membro è un'istanza posseduta dal giocatore, attiva e appartenente alla run corrente;
- la stessa istanza non può comparire due volte, mentre più istanze della stessa specie sono ammesse;
- l'**ordine** dei membri conta: è l'ordine di entrata in battaglia.

Il team si può modificare liberamente tra un encounter e l'altro (R9). Il giocatore può salvare più team per run (valore iniziale: massimo 5).

3.7 Encounter e stage

Gli encounter disponibili sono definiti da **template statici** caricati come seed (R12). Ogni stage ha tre template normali e un boss. Un template descrive le caratteristiche dell'encounter (stage, elemento prevalente, intervallo di livello, pesi delle rarità) ma non il mostro esatto: il selvatico viene estratto dal catalogo all'avvio della battaglia, rispettando quelle caratteristiche. Il giocatore vede quindi cosa aspettarsi senza conoscere l'avversario preciso.

- Sono disponibili i template degli stage già raggiunti, così il giocatore può tornare negli stage precedenti per fare esperienza.
- Il boss dello stage corrente si sblocca dopo **2 vittorie** nello stage (valore iniziale).
- Il boss ha una specie prefissata: `rare` negli stage 1–3, `legendary` di elementi diversi negli stage 4–7, **Omnivoid** (chaos, legendary) nello stage 8.
- Ogni boss si può battere **una volta per run** (R13): la vittoria dà il boss come ricompensa e fa salire lo stage.

Selvatico	Livello (stage s)	Esempio stage 8
Normale	casuale tra $3s-2$ e $3s$	22–24
Boss	$3s+2$	26

Tabella 2. Livelli dei selvatici (valori iniziali).

3.8 Battaglia

La battaglia è **interattiva a turni** (R3) nel formato **a staffetta** (R4): combatte un solo mostro del team alla volta, nell'ordine del team, contro il selvatico, che conserva i danni ricevuti per tutta la battaglia. Tutti i mostri iniziano con gli HP pieni (R10).

Azioni

A ogni turno il giocatore sceglie una delle tre azioni (R5):

Azione	Effetto
Attacco	Il mostro attivo attacca il selvatico.
Cambio	Un altro membro del team, non KO, prende il posto del mostro attivo. Il cambio consuma il turno.
Fuga	Tenta di chiudere l'encounter. Riesce con una probabilità che dipende dalla velocità (R7); se fallisce, il turno è perso.

Non esiste un'azione di cattura: la ricompensa è legata alla vittoria (R6).

Svolgimento del turno

Ogni richiesta del giocatore risolve un **round completo**: l'azione del giocatore e quella del selvatico, che attacca sempre.

1. Cambio e fuga si risolvono per primi.
2. Gli attacchi si risolvono in ordine di velocità effettiva; a parità decide il caso.
3. Se il mostro attivo va KO entra automaticamente il successivo ancora in piedi.
4. La battaglia termina con la vittoria (selvatico KO), con la sconfitta (tutto il team KO) o con una fuga riuscita.

Formule

Valori iniziali, da bilanciare con uno script di simulazione:

$$
\begin{aligned}
HP_{\max} &= 2\,hp_{\text{eff}} + 10 && \times 1{,}5 \text{ selvatico},\ \times 2{,}5 \text{ boss} \\
D &= \max\left(1,\ \text{round}\left(\frac{atk^2}{atk + def} \cdot 0{,}6 \cdot m_{\text{el}} \cdot v\right)\right) && v \sim U[0{,}9;\ 1{,}1] \\
m_{\text{el}} &\in \{1{,}5;\ 1;\ 0{,}67\} \\
p_{\text{fuga}} &= \min\left(0{,}95,\ \frac{spd_a}{spd_a + spd_w} + 0{,}25\right) \\
XP &= 10 \cdot L_w \cdot f_r \\
XP_{L \to L+1} &= 15\,L && L \leq 50
\end{aligned}
$$

Legenda: *atk* e *def* sono attacco dell'attaccante e difesa del difensore; m_{el} vale 1,5 se l'attacco è forte, 1 se neutro, 0,67 se debole; spd_a e spd_w sono le velocità del mostro attivo e del selvatico; L_w è il livello del selvatico e f_r vale 1, 1,5 o 2 per common, rare e legendary.

Gli HP del selvatico sono moltiplicati perché combatte da solo contro fino a tre avversari.

Abilità in battaglia

Le abilità di entrambi i contendenti si applicano secondo il loro `trigger`. Gli effetti *continui* (riduzione del danno, aumento del danno, schivata) valgono finché la condizione è vera; gli effetti *puntuali* (`heal`, `revive`) scattano una sola volta per battaglia.

Trigger	Quando si applica
`always`	Per tutta la battaglia
`first_turn`	Alla prima azione del mostro
`hp_below_50`	Con HP sotto il 50% (effetti puntuali: la prima volta che scendono sotto)
`hp_above_50`	Con HP sopra il 50%
`on_hit`	Quando il mostro subisce un attacco
`on_attack`	Quando il mostro attacca
`vs_faster`	Contro un avversario più veloce
`once`	Una volta per battaglia, all'evento previsto dall'effetto

Esiti

Esito	Ricompensa	Effetto sulla run
`victory`	Il selvatico entra in collezione al proprio livello; XP a tutti i membri del team (R11)	Continua; se boss, stage +1 o run vinta
`defeat`	Nessuna	Run persa: reset completo
`fled`	Nessuna	Continua
`expired`	Nessuna (battaglia inattiva da 30 minuti)	Continua
`interrupted`	Nessuna (riavvio del servizio)	Continua

Il mostro ottenuto entra in collezione al **livello del selvatico**, non a livello 1: in una struttura a run un mostro di livello 1 trovato allo stage 6 sarebbe inutilizzabile e non premierebbe la vittoria.

3.9 Run: fine, reset e resa

- **Sconfitta** (R14): la run diventa `lost` e parte il reset. Le istanze della run passano ad `archived`, i team della run vengono eliminati, il profilo riceve una nuova run con un nuovo pool di starter. Account, storico e run passate non vengono toccati.
- **Resa** (R17): il giocatore può abbandonare la run in qualsiasi momento fuori da una battaglia; la run diventa `abandoned` con lo stesso reset.
- **Vittoria finale** (R16): battuto il boss dello stage 8, la run diventa `won`. Il giocatore inizia una nuova run quando vuole; l'avvio esegue lo stesso reset.
- **Run passate** (R15): restano consultabili run, collezione archiviata e storico, così il giocatore può capire dove ha sbagliato.

4 Architettura

4.1 Stile architetturale

Il backend segue uno stile a **microservizi**: ogni componente con uno scopo chiaro diventa un servizio Quarkus indipendente, con il proprio database e il proprio ciclo di build e deploy. La scelta risponde direttamente al requisito di applicazione distribuita e permette a ogni membro del gruppo di essere responsabile di uno o due servizi, cosa utile anche per la valutazione individuale.

La consegna dell'architettura chiede uno schizzo dei componenti; i componenti individuati qui coincidono con i servizi, quindi lo schizzo e l'implementazione restano allineati.

4.2 Vista dei componenti

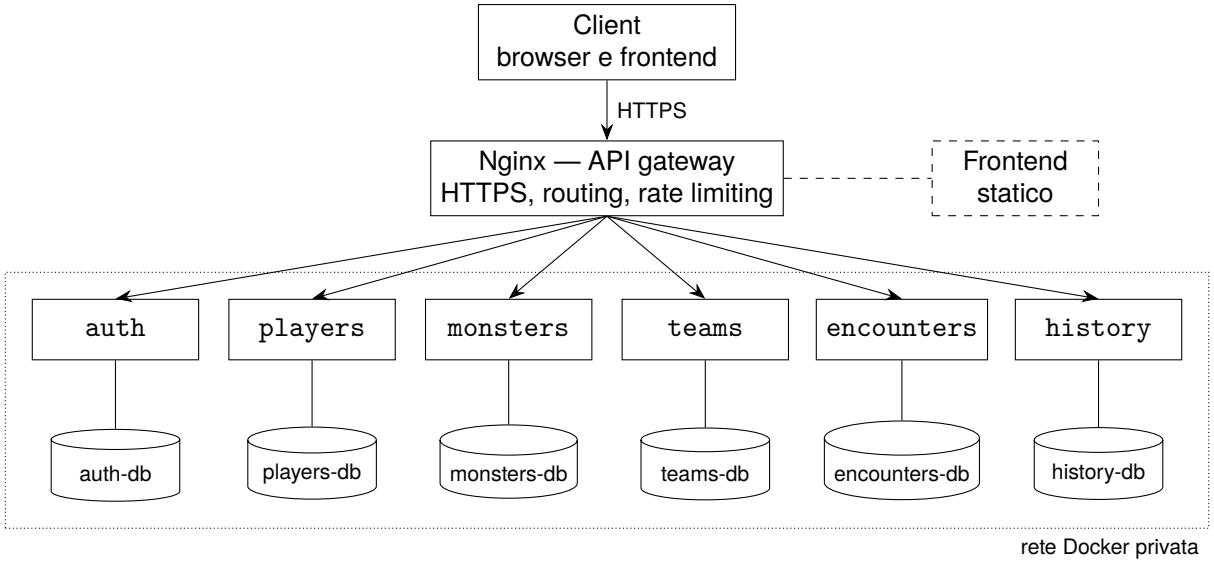


Figura 3. Vista dei componenti. Solo il gateway espone una porta; servizi e database sono raggiungibili solo sulla rete interna.

4.3 Responsabilità dei servizi

Servizio	Responsabilità	Dati	User stories
`auth`	Registrazione, login, token dei giocatori e dei servizi	`accounts`, `refresh_tokens`	US-01, 02, 05
`players`	Profilo, pool degli starter, run corrente	`players`	US-03, 04, 06, 07
`monsters`	Catalogo, abilità, elementi, immagini; collezione per run, livelli ed XP	`monsters`, `abilities`, `elements`, `owned_monsters`	US-09, 10, 11, 12, 13
`teams`	Creazione, modifica, validazione dei team	`teams`, `team_slots`	US-14, 15, 16
`encounters`	Catalogo degli encounter, run, battaglie, orchestrazione di fine run	`encounter_templates`, `runs`; battaglie in memoria	US-17–22
`history`	Storico persistente e integrità dei salvataggi	`encounter_records`	US-23, 24, 25

4.4 Gateway

Nginx è l'unico punto di ingresso (A3). Svolge quattro compiti: instradare le richieste verso il servizio competente, terminare HTTPS, limitare la frequenza delle richieste sugli endpoint di autenticazione e servire il frontend statico. Non contiene logica applicativa né verifica i token: questo compito resta ai servizi.

Percorso pubblico	Servizio
`/api/auth/*`	`auth`
`/api/players/*`	`players`
`/api/monsters/*`, `/api/abilities/*`, `/api/elements`, `/api/collection/*`, `/images/*`	`monsters`
`/api/teams/*`	`teams`
`/api/encounters/*`, `/api/runs/*`, `/api/battles/*`	`encounters`
`/api/history/*`	`history`
`/`	frontend

I percorsi `/internal/*` non sono mai instradati.

4.5 Comunicazione tra servizi

I servizi comunicano solo tramite **REST sincrono** con i REST client tipizzati di Quarkus (A5). Le chiamate interne sono protette dalle annotazioni di SmallRye Fault Tolerance (`@Timeout`, `@Retry`) e sono **idempotenti**: ogni chiamata che modifica dati porta una chiave (`run_id` o `encounter_id`) che permette di ripeterla senza effetti doppi.

Il grafo delle dipendenze non ha cicli tra chiamate sincrone nello stesso flusso: `players` chiama `encounters` solo all'apertura della run, `encounters` chiama `players` solo alla chiusura.

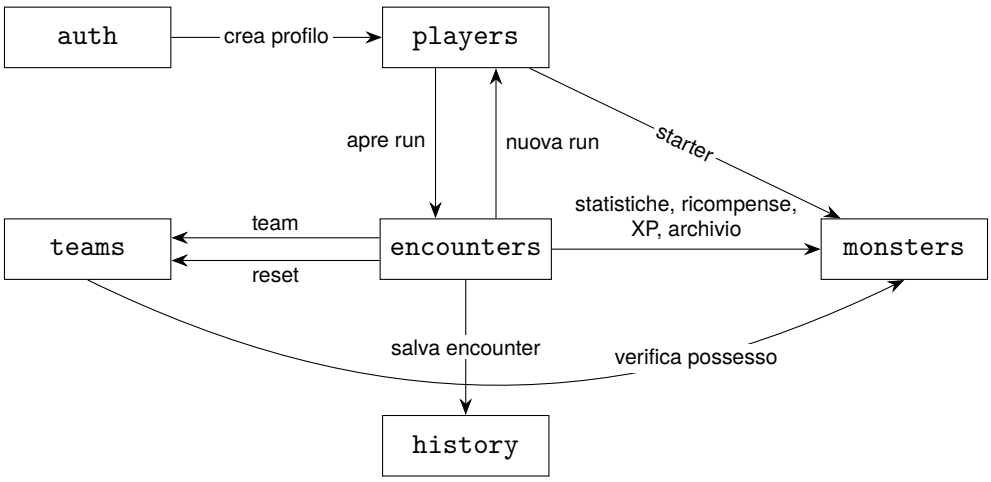


Figura 4. Dipendenze tra servizi. Ogni freccia è una chiamata a un endpoint `/internal/*` con token di servizio. `history` non chiama nessuno; `encounters` orchestra la fine delle battaglie.

4.6 Autenticazione e autorizzazione

Il servizio `auth` firma i token con una chiave privata RSA (A6); gli altri servizi montano solo la chiave pubblica e verificano i token localmente, senza chiamare `auth` a ogni richiesta.

- **Access token**: JWT RS256 con `sub` = `player_id` e ruolo `player`, durata 15 minuti.
- **Refresh token**: valore casuale, salvato in `auth` solo come hash, durata 7 giorni, revocabile con il logout.
- **Token di servizio**: JWT con ruolo `service`, ottenuto da ciascun servizio presso `auth` con credenziali proprie (*client credentials*) e riusato fino alla scadenza.
- **Autorizzazione per risorsa**: il `player_id` si legge sempre dal token, mai da parametri del client. Un giocatore non può quindi indicare le risorse di un altro.

Gli endpoint `/internal/*` hanno due livelli di protezione (A7): non sono raggiungibili dall'esterno perché il gateway non li instrada, e richiedono comunque un token con ruolo `service`.

4.7 Stack tecnologico

Ambito	Tecnologia	Motivo
Linguaggio e framework	Java 21, Quarkus (LTS 3.x)	Scelta del gruppo; avvio rapido, Dev Services, estensioni pronte
API REST	Quarkus REST + Jackson, SmallRye OpenAPI	Specifica OpenAPI generata dal codice
Persistenza	Hibernate ORM con Panache, Flyway	Modello più documentato; migrazioni versionate
Database	PostgreSQL	Relazionale, supporto JSONB per log e snapshot
Sicurezza	SmallRye JWT, bcrypt, Hibernate Validator	Verifica locale dei token; hash robusto; validazione dichiarativa
Chiamate interne	REST Client, SmallRye Fault Tolerance	Client tipizzati; timeout e retry dichiarativi
Gateway	Nginx	Configurazione dichiarativa, nessun codice
Build e deploy	Maven, Docker, docker compose	Un comando per buildare, uno per avviare
Test	JUnit 5, RestAssured, WireMock, Locust	Unit, API, integrazione, performance

5 Servizi

Per ogni servizio sono riportati endpoint e dati. Tutti gli endpoint pubblici, tranne registrazione e login, richiedono l'intestazione `Authorization: Bearer` con un access token valido. Le chiavi di idempotenza degli endpoint interni sono indicate nella descrizione.

5.1 `auth`

Metodo	Endpoint	Descrizione
POST	`/api/auth/register`	Crea l'account da `username` e `password` e restituisce il `player_id`. Crea il profilo su `players`; se la chiamata fallisce, l'account viene annullato.
POST	`/api/auth/login`	Restituisce access e refresh token. Con credenziali errate restituisce un errore generico, senza indicare quale campo è sbagliato.
POST	`/api/auth/refresh`	Emette un nuovo access token da un refresh token valido.
POST	`/api/auth/logout`	Revoca il refresh token.
POST	`/internal/auth/service-token`	Emette un token di servizio da `client_id` e `client_secret`.

Campo	Tipo	Note
Tabella `accounts`		
`player_id`	UUID, PK	Stesso id del profilo
`username`	varchar(20), unique	3–20 caratteri
`password_hash`	varchar	bcrypt con salt; la password in chiaro non viene mai salvata
`created_at`, `last_login`	timestamp	
Tabella `refresh_tokens`		
`token_hash`	char(64), PK	SHA-256 del token
`player_id`	UUID	
`expires_at`	timestamp	
`revoked`	boolean	

La chiave privata di firma esiste solo in `auth`. Le credenziali dei servizi sono passate come variabili d'ambiente.

5.2 `players`

Metodo	Endpoint	Descrizione
GET	`/api/players/me`	Profilo, run corrente e stato degli starter.
PATCH	`/api/players/me`	Modifica nome visualizzato e avatar (US-04).
GET	`/api/players/me/starters`	Pool di 10 common della run corrente.
POST	`/api/players/me/starters`	Sceglie 3 starter distinti dal pool; li aggiunge in `monsters` e apre la run in `encounters`.
POST	`/internal/players`	Crea il profilo; idempotente su `player_id`.
POST	`/internal/players/{pid}/new-run`	Assegna una nuova run e un nuovo pool. Riceve la run conclusa e non fa nulla se non è quella corrente: per questo è idempotente.

Campo	Tipo	Note
Tabella `players`		
`player_id`	UUID, PK	
`display_name`	varchar(30)	Inizialmente uguale allo username
`avatar`	varchar, nullable	
`current_run_id`	UUID	Generato dal servizio a ogni nuova run
`starter_pool`	int[10]	Specie common estratte per la run corrente
`starters_chosen`	boolean	`true` dopo la scelta; torna `false` al reset
`created_at`	timestamp	

Lo stage non è nel profilo ma nella run (`encounters`), perché si azzera a ogni run.

5.3 monsters

Metodo	Endpoint	Descrizione
GET	`/api/monsters`	Catalogo paginato, filtrabile per `element` e `rarity` (US-10).
GET	`/api/monsters/{id}`	Dettaglio della specie con abilità (US-11).
GET	`/api/abilities`, `/api/abilities/{id}`	Catalogo delle abilità.
GET	`/api/elements`	Ciclo degli elementi.
GET	`/api/collection`	Collezione della run corrente; con `run_id` quella archiviata di una run passata, solo del giocatore stesso (US-09).
GET	`/api/collection/{owned_id}`	Istanza con livello, XP, statistiche effettive, provenienza (US-11).
GET	`/images/monsters/{NNN}.png`	Sprite statico.
POST	`/internal/collections/{pid}/starters`	Aggiunge i 3 starter; idempotente su `run_id`.
POST	`/internal/collections/{pid}/monsters`	Aggiunge il mostro vinto al livello indicato; idempotente su `encounter_id` (US-12).
POST	`/internal/collections/{pid}/xp`	Assegna XP alle istanze e calcola i level up; idempotente su `encounter_id`.
GET	`/internal/collections/{pid}/instances`	Istanze richieste con statistiche effettive e abilità.
GET	`/internal/monsters/{id}/stats`	Statistiche di una specie a un dato livello, per il selvatico.
POST	`/internal/collections/{pid}/archive`	Archivia le istanze di una run; idempotente su `run_id`.

Campo	Tipo	Note
Tabelle di catalogo `monsters`, `abilities`, `elements`		
Struttura identica ai file di seed; caricate da una migrazione Flyway e mai modificate a runtime. `monsters.ability_id` è una chiave esterna reale, perché le tabelle sono nello stesso database.		
Tabella `owned_monsters`		
`owned_id`	UUID, PK	Referenziato da team e storico
`player_id`	UUID	Riferimento logico a `players`
`run_id`	UUID	Run di appartenenza
`monster_id`	int, FK	Specie
`level`	smallint	1–50
`xp`	int	XP accumulata nel livello corrente
`source`	enum	`starter`, `encounter`, `boss`
`encounter_id`	UUID, unique, nullable	Encounter di provenienza; garantisce l'idempotenza
`status`	enum	`active`, `archived`
`obtained_at`	timestamp	
Tabella `xp_grants`		
`encounter_id`	UUID, PK	XP già assegnata per quell'encounter
`granted_at`	timestamp	

La formula delle statistiche effettive esiste solo in questo servizio; `teams` ed `encounters` la usano tramite gli endpoint interni.

5.4 `teams`

Metodo	Endpoint	Descrizione
GET	`/api/teams`	Team della run corrente.
POST	`/api/teams`	Crea un team con nome e membri ordinati, dopo averlo validato (US-14).
GET	`/api/teams/{id}`	Dettaglio del team.
PUT	`/api/teams/{id}`	Modifica membri e ordine (US-15).
DELETE	`/api/teams/{id}`	Elimina il team.
POST	`/api/teams/validate`	Valida una composizione senza salvarla; restituisce l'elenco degli errori (US-16).
GET	`/internal/teams/{id}`	Team rivalidato, con le istanze, per l'avvio di una battaglia.
POST	`/internal/teams/{pid}/reset`	Elimina i team di una run; idempotente su `run_id`.

Campo	Tipo	Note
Tabella `teams`		
`team_id`	UUID, PK	
`player_id`, `run_id`	UUID	
`name`	varchar(30)	
`created_at`, `updated_at`	timestamp	
Tabella `team_slots`		
`team_id`	UUID, FK	PK insieme a `position`
`position`	smallint	1–3, ordine di entrata in battaglia
`owned_id`	UUID	Unico all'interno del team

La validazione chiede a `monsters` le istanze indicate e controlla: numero di membri tra 1 e 3, possesso, stato `active`, appartenenza alla run corrente, assenza di istanze ripetute. Un team valido al salvataggio viene rivalidato all'avvio di ogni battaglia.

5.5 encounters

Metodo	Endpoint	Descrizione
GET	`/api/encounters`	Encounter disponibili: template degli stage raggiunti e boss dello stage corrente, se sbloccato (US-17).
GET	`/api/encounters/{template_id}`	Caratteristiche del template.
GET	`/api/runs`	Run del giocatore, correnti e passate, con esito e stage raggiunto.
GET	`/api/runs/current`	Stage, vittorie nello stage, boss battuti.
GET	`/api/runs/{run_id}`	Dettaglio di una run.
POST	`/api/runs/current/abandon`	Resa: chiude la run ed esegue il reset.
POST	`/api/runs`	Nuova run dopo una run vinta; esegue il reset.
POST	`/api/battles`	Avvia una battaglia da template e team; 409 se ne esiste già una (US-18).
GET	`/api/battles/current`	Stato della battaglia: mostro attivo, HP, turno, selvatico.
POST	`/api/battles/current/actions`	Esegue un'azione (`attack`, `switch`, `flee`) e restituisce l'esito del round (US-19, US-21, US-22). Alla fine della battaglia restituisce esito e ricompense (US-20).
POST	`/internal/runs`	Apre una run allo stage 1; idempotente su `run_id`.

Campo	Tipo	Note
Tabella `encounter_templates` (seed)		
`template_id`	int, PK	
`stage`	smallint	1–8
`name`, `description`	varchar, text	
`element`	varchar, nullable	Elemento prevalente; `null` = qualsiasi
`rarity_weights`	JSONB	Probabilità di estrazione per rarità
`level_min`, `level_max`	smallint	
`is_boss`	boolean	
`boss_monster_id`	int, nullable	Specie del boss
Tabella `runs`		
`run_id`	UUID, PK	Generato da `players`
`player_id`	UUID	Al massimo una run `active` per giocatore (indice parziale)
`status`	enum	`active`, `won`, `lost`, `abandoned`
`stage`	smallint	1–8
`wins_in_stage`	smallint	Per lo sblocco del boss
`bosses_defeated`	int[]	Stage il cui boss è stato battuto
`active_battle_id`	UUID, nullable	Battaglia in corso
`reset_pending`	boolean	Reset da completare (Sezione 7)
`version`	int	Lock ottimistico (`@Version`)
`started_at`, `ended_at`	timestamp	

Battaglie in memoria

Lo stato delle battaglie in corso è tenuto in memoria (R19), in una mappa indicizzata per giocatore: snapshot del team, HP correnti, mostro attivo, effetti attivi, turni giocati, seed del generatore casuale e istante dell'ultima azione. Le azioni di una stessa battaglia sono elaborate in modo sequenziale.

La run, invece, è persistita: all'avvio di una battaglia il suo id viene salvato in `runs.active_battle_id`. Questo permette di gestire due casi:

- **Scadenza**: un job eseguito ogni minuto chiude come `expired` le battaglie inattive da più di 30 minuti (R18).
- **Riavvio**: all'avvio il servizio cerca le run con `active_battle_id` valorizzato ma senza battaglia in memoria e le chiude come `interrupted`.

In entrambi i casi l'encounter viene registrato nello storico e la run continua.

5.6 `history`

Metodo	Endpoint	Descrizione
GET	`/api/history`	Encounter passati del giocatore, paginati e filtrabili per `run_id` ed esito (US-23).
GET	`/api/history/{encounter_id}`	Dettaglio completo con team, selvatico e log dei turni (US-24).
POST	`/internal/history`	Registra un encounter concluso; idempotente su `encounter_id`.

Campo	Tipo	Note
Tabella `encounter_records`		
`encounter_id`	UUID, PK	
`player_id`, `run_id`	UUID	
`template_id`, `stage`	int, smallint	
`wild`	JSONB	Specie, livello e statistiche del selvatico
`team_snapshot`	JSONB	Istanze con livello e statistiche al momento della battaglia
`outcome`	enum	`victory`, `defeat`, `fled`, `expired`, `interrupted`
`reward_owned_id`	UUID, nullable	Istanza ottenuta
`xp_gained`	int	
`turns`	JSONB	Log dei turni: azione, danni, abilità attivate
`rng_seed`	bigint	Permette di rigiocare la battaglia
`started_at`, `ended_at`	timestamp	
`prev_hash`, `hash`	char(64)	Catena di integrità (Sezione 8)

Lo storico salva uno snapshot completo (R20): il dettaglio di un encounter passato resta leggibile anche dopo che le istanze sono state archiviate o hanno cambiato livello.

6 Flussi principali

Nei diagrammi il gateway è omesso: tutte le richieste del client passano da Nginx. Le frecce continue sono richieste, quelle tratteggiate risposte.

6.1 Registrazione e scelta degli starter

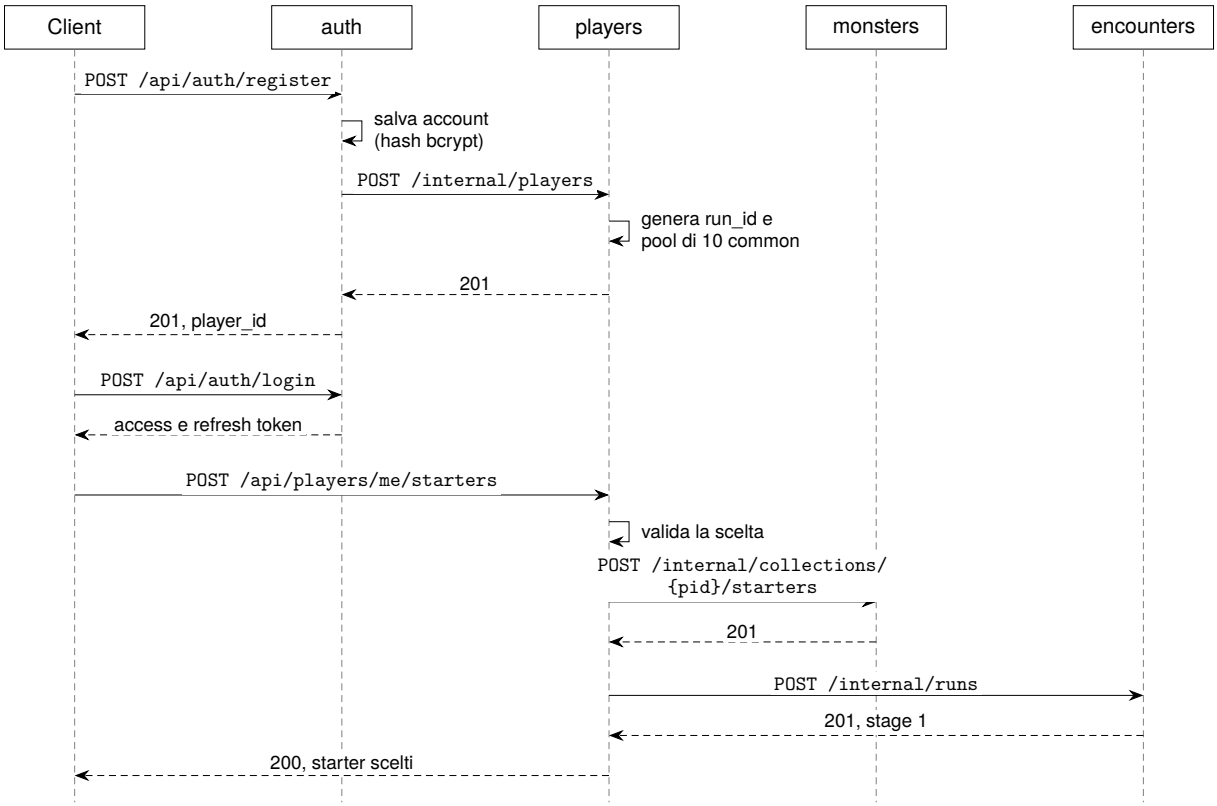


Figura 5. Registrazione, login e scelta degli starter.

Se la creazione del profilo fallisce, `auth` annulla l'account appena creato e restituisce un errore: non restano account senza profilo. La scelta degli starter segna `starters_chosen` solo dopo che entrambe le chiamate interne sono riuscite; essendo idempotenti, una richiesta ripetuta dal client completa il lavoro senza duplicati.

6.2 Avvio e svolgimento di una battaglia

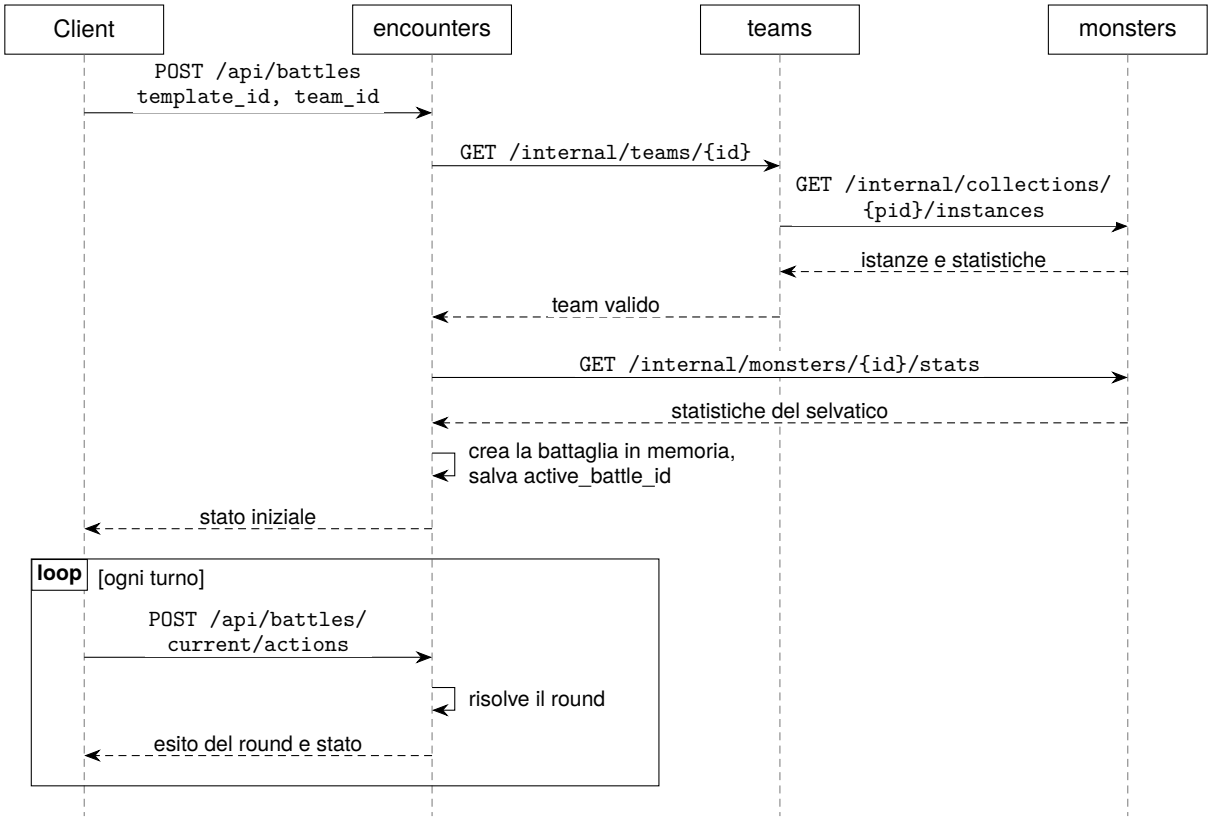


Figura 6. Avvio di una battaglia e ciclo dei turni.

Durante i turni `encounters` non chiama altri servizi: tutto quello che serve è nello snapshot preso all'avvio. Il costo di una mossa è quindi una sola richiesta HTTP, che non dipende dalla disponibilità degli altri servizi.

6.3 Fine della battaglia e reset della run

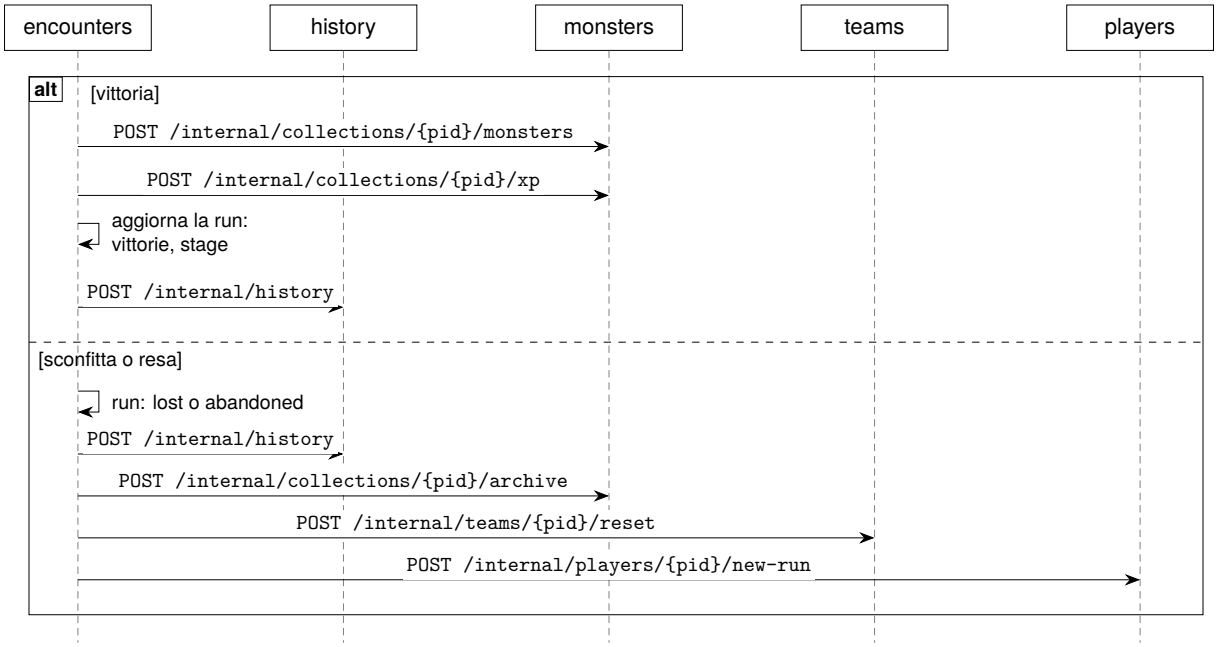


Figura 7. Fine di una battaglia: ricompense in caso di vittoria, reset della run in caso di sconfitta o resa. Le risposte sono omesse.

Il reset è una **saga orchestrata** da `encounters` (Sezione 7). La resa segue lo stesso ramo, senza il passo sullo storico se non c'era una battaglia in corso.

7 Dati e consistenza

7.1 Un database per servizio

Ogni servizio ha il proprio container PostgreSQL e le proprie credenziali (A4); nessun servizio può leggere i dati di un altro. Ne derivano due conseguenze:

- i riferimenti tra dati di servizi diversi (per esempio `owned_monsters.player_id` o `team_slots.owned_id`) sono **logici**: si salva l'identificativo, ma il database non ne garantisce l'esistenza. Il controllo è fatto dal servizio, tramite le chiamate interne;
- non esistono transazioni che coinvolgono più servizi: la consistenza tra servizi va gestita esplicitamente.

7.2 Consistenza tra servizi

Le operazioni che toccano più servizi sono tre: registrazione, scelta degli starter e fine della battaglia. Tutte seguono le stesse regole:

1. ogni chiamata interna che modifica dati è **idempotente** grazie a una chiave naturale dell'operazione (`player_id`, `run_id`, `encounter_id`), protetta da un vincolo `unique` nel database del servizio chiamato;
2. in caso di errore temporaneo le chiamate sono ripetute automaticamente, con `@Retry` e `@Timeout`;

3. lo stato locale del chiamante viene aggiornato prima delle chiamate, così da sapere sempre cosa resta da fare.

Per il reset della run, `encounters` imposta `status` a `lost` (o `abandoned`) e `reset_pending` a `true`, poi esegue in ordine archiviazione della collezione, reset dei team e nuova run sul profilo. Quando tutti i passi riescono, `reset_pending` torna `false`. Se un passo fallisce in modo definitivo, il flag resta attivo e il reset riprende alla richiesta successiva del giocatore: grazie all'idempotenza, i passi già eseguiti non hanno effetti doppi.

7.3 Migrazioni e seed

Lo schema di ogni database è definito da migrazioni Flyway versionate nel repository del servizio. I dati statici (catalogo dei mostri, abilità, elementi, template degli encounter) sono caricati da migrazioni dedicate e non sono modificabili tramite API. I file di seed dei mostri sono generati dalla pipeline `tools/monster-gen/` e non vanno modificati a mano.

7.4 Immagini

Gli sprite dei mostri sono file statici serviti da `monsters`; nel database c'è solo il percorso (`image_url`). Il frontend li mostra ingranditi senza sfocatura, con la proprietà CSS `image-rendering: pixelated`.

8 Sicurezza

Le misure sono progettate fin da ora, anche se US-05 e US-25 appartengono alla consegna finale: introdurle più tardi richiederebbe di modificare tutti i servizi.

Minaccia	Contromisura	Dove
Intercettazione del traffico	HTTPS terminato sul gateway (certificato self-signed in sviluppo)	Nginx
Furto del database delle credenziali	Password salvate solo come hash bcrypt con salt; refresh token salvati come hash	`auth`
Furto di un token	Access token di 15 minuti; refresh token revocabile	`auth`
Accesso ai dati di un altro giocatore	`player_id` letto solo dal token; ogni risorsa verificata contro il proprietario	tutti
Chiamata diretta a endpoint interni	`/internal/*` non instradati; ruolo `service` obbligatorio	Nginx, tutti
Brute force su login e registrazione	Rate limiting per indirizzo IP	Nginx
Injection e input malformati	Validazione dichiarativa (Hibernate Validator); query parametrizzate tramite ORM	tutti
Imbrogli sul gioco	Regole, estrazioni casuali e ricompense solo nel backend; nessun endpoint per aggiungere mostri o XP	`encounters`, `monsters`

Minaccia	Contromisura	Dove
Manomissione dello storico	Tabella append-only (l'utente applicativo non ha `UPDATE` né `DELETE`); catena di hash SHA-256 per giocatore; seed salvato per rigiocare e verificare una battaglia	`history`
Perdita dei salvataggi	Dati persistenti su volumi Docker; le run concluse vengono archiviate, non cancellate	tutti
Segreti nel repository	Credenziali e chiavi solo da variabili d'ambiente	docker compose

La catena di hash funziona così: ogni record contiene l'hash del record precedente dello stesso giocatore e il proprio hash, calcolato su `prev_hash` e sul contenuto del record. Modificare o cancellare un record rompe la catena da quel punto in poi, e la manomissione diventa rilevabile.

L'analisi di sicurezza completa, con gli strumenti che verranno presentati nel corso, è prevista per la consegna finale.

9 Deployment e test

9.1 Deployment

L'intero sistema si avvia con docker compose: Nginx, sei servizi Quarkus e sei container PostgreSQL, collegati da una rete Docker privata. Solo Nginx espone una porta all'esterno.

Percorso	Contenuto
`backend/pom.xml`	pom aggregatore: versione di Quarkus, Java 21, moduli
`backend/<servizio>/`	progetto Maven del servizio, `Dockerfile`
`backend/<servizio>/src/main/resources/db/migration/`	migrazioni Flyway
`gateway/nginx.conf`	configurazione del gateway
`frontend/`	frontend statico
`docker-compose.yml`	topologia completa

Tabella 3. Organizzazione del repository.

I servizi non condividono codice (A9): ognuno definisce i propri DTO. La duplicazione è limitata e viene accettata in cambio dell'indipendenza: una modifica a un servizio non obbliga a ricompilare gli altri.

9.2 Strategia di test

Livello	Oggetto	Strumenti
Unitario	Formule, regole di validazione del team, motore di battaglia con seed fisso (risultati deterministici)	JUnit 5
API del servizio	Endpoint, codici di risposta, autorizzazione	`@QuarkusTest`, RestAssured, Dev Services
Integrazione	Chiamate verso altri servizi, retry, idempotenza	WireMock, mock dei REST client
End-to-end	Flusso completo tramite gateway	docker compose, RestAssured
Performance	Login, catalogo, battaglie concorrenti	Locust

Il motore di battaglia usa un generatore casuale con seed: con lo stesso seed e le stesse azioni produce sempre lo stesso risultato. Questo rende i test deterministici e permette di verificare a posteriori una battaglia registrata nello storico.

10 Registro delle decisioni

Per ogni decisione sono riportate la scelta, le alternative considerate e la motivazione.

10.1 Architettura

ID	Decisione	Alternative scartate	Motivazione e conseguenze
A1	Quarkus, Java 21	—	Scelta del gruppo. Dev Services, estensioni pronte per REST, JWT, ORM, fault tolerance.
A2	6 servizi	7 (catalogo separato dalla collezione); 5 (storico dentro `encounters`); 4 accorpati	Confini chiari e un servizio per responsabilità; lavoro divisibile tra i membri. Costo: più container e chiamate interne.
A3	Gateway Nginx	Gateway Quarkus custom; Traefik; nessun gateway	Solo configurazione: routing, HTTPS, rate limiting. Senza gateway servirebbero CORS e HTTPS su ogni servizio.
A4	PostgreSQL, un container per servizio	Un PostgreSQL con database separati; PostgreSQL + MongoDB per lo storico	Isolamento reale dei dati e dei guasti; una sola tecnologia di database. Costo: più container, comunque leggeri.
A5	Solo REST sincrono	REST + broker di eventi (RabbitMQ, Kafka)	KISS: un solo paradigma, test semplici. La resilienza si ottiene con retry e idempotenza invece che con un broker.

ID	Decisione	Alternative scartate	Motivazione e conseguenze
A6	JWT RS256 firmati da `auth`	Keycloak con OIDC; token opachi verificati da `auth`	Verifica locale senza chiamate ad `auth`; nessun collo di bottiglia. La revoca è gestita con token brevi e refresh token revocabili.
A7	`/internal/*` non instradati + token di servizio	Solo isolamento di rete; mTLS	Doppia difesa a costo moderato. mTLS resta un'estensione possibile.
A8	Hibernate Panache + Flyway	Hibernate Reactive; JDBC o jOOQ	Modello più semplice e documentato; il vantaggio del reattivo non è visibile a questa scala.
A9	Maven, pom aggregatore, nessun codice condiviso	Modulo `common` condiviso; progetti indipendenti; Gradle	Un comando di build senza accoppiare i servizi tramite una libreria comune.

10.2 Regole di gioco

ID	Decisione	Alternative scartate	Motivazione e conseguenze
R1	Roguelike a run	Progressione continua senza reset	Ogni run ha inizio e fine; la sconfitta ha un peso reale. Introduce l'entità run e il reset tra servizi.
R2	8 stage con boss finale	Zone per elemento aperte; obiettivo di completare il catalogo	Obiettivo chiaro e difficoltà crescente, come nell'esempio del docente.
R3	Battaglia interattiva a turni	Simulata in una chiamata; simulata con tattica iniziale; formula a punteggio	Il giocatore prende decisioni a ogni turno; copre anche US-21 e US-22. Richiede stato tra richieste.
R4	1v1 a staffetta	Tutto il team contro il selvatico	Regole di danno semplici; l'ordine del team diventa una scelta strategica.
R5	Attacco, cambio, fuga	Anche un'azione di cattura	La cattura è stata esclusa per coerenza con R6.
R6	Vittoria = selvatico in collezione	Ottenimento con probabilità per rarità; cattura come azione	Regola semplice; ogni vittoria fa crescere la collezione.
R7	Fuga con probabilità basata sulla velocità	Fuga sempre riuscita	La velocità conta anche fuori dall'attacco; la fuga ha un rischio.
R8	Team da 1 a 3, copie della stessa specie ammesse, nessun altro vincolo	Massimo 4 o 6; specie distinte e massimo un legendary; budget di rarità	Coincide con i 3 starter; regole minime e facili da validare.

ID	Decisione	Alternative scartate	Motivazione e conseguenze
R9	Team modificabile tra un encounter e l'altro	Team bloccato per tutta la run	Libertà strategica; i mostri appena ottenuti si usano subito.
R10	HP pieni a ogni encounter	HP persistenti nella run, con o senza cura	Ogni encounter è indipendente; stato della run più semplice.
R11	XP a tutto il team	Solo a chi è sceso in campo; nessun livello	Semplice e generoso; permette di far crescere i mostri in coda al team.
R12	Template statici, selvatico estratto	Encounter generati per giocatore a rotazione; selvatico fisso	Dati semplici, varietà nelle battaglie, nessuno stato per giocatore.
R13	Boss battibile una volta per run, dà il boss	Boss ripetibile; boss non ottenibile	Rende i legendary trofei della run e impedisce di accumularli.
R14	Sconfitta: reset completo	Nessuna penalità; perdita di XP; mostri a riposo; reset del solo stage; reset dello stage e dei mostri della run	Scelta del gruppo per un roguelike puro. Richiede il reset coordinato di quattro servizi.
R15	Istanze archiviate, non cancellate	Cancellazione	Le run passate restano consultabili; nessuna perdita di dati irreversibile.
R16	Run vinta, poi nuova run	Modalità libera dopo la vittoria	Ciclo coerente: ogni run ha un inizio e una fine.
R17	Resa ammessa	Nessuna resa	Utile con un pool di starter sfortunato; riusa la logica di reset.
R18	Una battaglia attiva, scadenza dopo 30 minuti	Nessuna scadenza; battaglie in parallelo	Evita battaglie abbandonate per sempre e conflitti sullo stesso mostro.
R19	Battaglia in memoria, run in PostgreSQL	Tutto in PostgreSQL; Redis; tutto in memoria	Battaglia veloce e semplice da gestire; la run sopravvive ai riavvii. Limite: battaglie perse al riavvio e una sola istanza di `encounters`.
R20	Storico con riepilogo e log dei turni	Solo riepilogo	Dettaglio completo per US-24 e base per la verifica di integrità.

10.3 Modifiche alle decisioni precedenti

L'introduzione delle run modifica alcune decisioni del documento dell'area giocatori e mostri:

Prima	Ora
`Players.stage`	Spostato in `runs.stage`, nel servizio `encounters`
Starter scelti una sola volta (D7)	Una volta per run
Mostro ottenuto a livello 1 (US-12)	Al livello del selvatico
`OwnedMonsters` senza run	Aggiunti `run_id` e `status`
Progressi sempre conservati (US-03)	Conservati finché la run non termina; le run passate restano consultabili

11 Limiti noti e punti aperti

11.1 Limiti noti

- **Battaglie in memoria** (R19): un riavvio di `encounters` interrompe le battaglie in corso e il servizio non può essere replicato su più istanze. Run, stage e collezione non si perdono. Il passaggio a uno stato persistente non cambierebbe le API.
- **Consistenza eventuale durante il reset**: tra la fine di una run e il completamento del reset, alcuni servizi possono mostrare ancora i dati della run conclusa per un breve intervallo.
- **Comunicazione sincrona**: se un servizio chiamato è irraggiungibile, l'operazione che lo coinvolge fallisce dopo i retry; le battaglie in corso non ne sono toccate.

11.2 Punti aperti

- Bilanciamento di formule, livelli e soglie con uno script di simulazione.
- Definizione dei 32 template degli encounter e scelta dei boss degli stage 1–7.
- Dettaglio degli effetti `copy_ability`, `invert_advantage` e `random_boost`.
- Analisi di sicurezza completa per la consegna finale.
- Frontend: tecnologia e schermate.

A Tracciabilità delle user stories

Priorità: **P** primo prototipo, **F** consegna finale, **O** opzionale.

ID	Titolo	Pr.	Servizio	Endpoint principali
US-01	Creare un account	P	`auth`	POST `/api/auth/register`
US-02	Accedere (login)	P	`auth`	POST `/api/auth/login`
US-03	Ritrovare i progressi salvati	P	`players`	GET `/api/players/me`, GET `/api/runs/current`
US-04	Personalizzare il profilo	O	`players`	PATCH `/api/players/me`
US-05	Proteggere i dati dell'account	F	tutti	Sezione 8
US-06	Ricevere una collezione iniziale	P	`players`	GET, POST `/api/players/me/starters`

ID	Titolo	Pr.	Servizio	Endpoint principali
US-07	Scegliere il primo mostro	O	`players`	come US-06
US-08	Scegliere una fazione	O	—	non coperta
US-09	Vedere la collezione personale	P	`monsters`	GET `/api/collection`
US-10	Sfogliare la collezione globale	P	`monsters`	GET `/api/monsters`
US-11	Vedere i dettagli di un mostro	P	`monsters`	GET `/api/monsters/{id}`, GET `/api/collection/{owned_id}`
US-12	Ottenere un mostro guadagnato	P	`monsters`	interno, a fine battaglia
US-13	Sviluppare i mostri	O	`monsters`	livelli e XP, interno
US-14	Creare un team	P	`teams`	POST `/api/teams`
US-15	Modificare un team	P	`teams`	PUT `/api/teams/{id}`
US-16	Validare un team	P	`teams`	POST `/api/teams/validate`
US-17	Sfogliare gli encounter	P	`encounters`	GET `/api/encounters`
US-18	Avviare un encounter	P	`encounters`	POST `/api/battles`
US-19	Risolvere un encounter	P	`encounters`	POST `/api/battles/current/actions`
US-20	Dettagli del risultato	P	`encounters`	risposta dell'ultima azione
US-21	Abbandonare un encounter	O	`encounters`	azione `flee`; resa della run
US-22	Scegliere le azioni	O	`encounters`	azioni `attack`, `switch`, `flee`
US-23	Vedere lo storico	P	`history`	GET `/api/history`
US-24	Dettagli di un encounter passato	P	`history`	GET `/api/history/{encounter_id}`
US-25	Proteggere i salvataggi	F	`history`, tutti	Sezione 8
US-26	Gestire gli amici	O	—	non coperta
US-27	Sfidare un amico	O	—	non coperta
US-28	Usare un'interfaccia semplice	P	frontend	tutte le API pubbliche

B Catalogo delle abilità

Valori da `backend/monsters/seed/abilities.json`. L'ambito indica se l'abilità è generica, legata a un elemento o esclusiva dei legendary.

ID	Nome	Ambito	Effetto	Valore	Trigger
1	Thick Hide	generica	`damage_reduction`	10	`always`
2	Quick Reflexes	generica	`dodge`	10	`always`

ID	Nome	Ambito	Effetto	Valore	Trigger
3	Second Wind	generica	heal	15	hp_below_50
4	Opening Strike	generica	damage_boost	20	first_turn
5	Stubborn	generica	damage_reduction	20	hp_below_50
6	Momentum	generica	speed_boost	15	always
7	Spiteful Thorns	generica	reflect	10	on_hit
8	Lullaby	dream	stun	15	on_attack
9	Lucid Shift	dream	dodge	20	hp_above_50
10	Resonance	echo	damage_boost	15	always
11	Feedback	echo	reflect	20	on_hit
12	Rewind	time	heal	25	hp_below_50
13	Haste	time	speed_boost	25	first_turn
14	Heavy Body	gravity	damage_reduction	20	always
15	Crushing Pull	gravity	damage_boost	25	vs_faster
16	Muffle	silence	weaken	15	first_turn
17	Unseen	silence	dodge	25	first_turn
18	Recall	memory	damage_boost	20	hp_below_50
19	Imprint	memory	ignore_ disadvantage	100	always
20	Entropy	chaos	random_boost	30	always
21	Wild Surge	chaos	damage_boost	40	hp_below_50
22	Dreamweaver	legendary	stun	35	on_attack
23	Endless Chorus	legendary	reflect	40	on_hit
24	Hourglass Reversal	legendary	revive	50	once
25	Event Horizon	legendary	damage_reduction	40	always
26	Absolute Quiet	legendary	weaken	35	always
27	Eternal Archive	legendary	copy_ability	100	first_turn
28	Paradox Engine	legendary	invert_advantage	100	always

Effetto	Significato in battaglia
`damage_boost`	Danno inflitto aumentato del valore (%)
`damage_reduction`	Danno subito ridotto del valore (%)
`dodge`	Probabilità (%) di evitare un attacco
`heal`	Recupera il valore (%) degli HP massimi, una volta
`speed_boost`	Velocità aumentata del valore (%)
`reflect`	Restituisce all'attaccante il valore (%) del danno subito
`stun`	Probabilità (%) che l'avversario salti la prossima azione
`weaken`	Attacco dell'avversario ridotto del valore (%)
`revive`	Una volta per battaglia, al KO torna con il valore (%) degli HP
`random_boost`	A ogni azione una statistica casuale aumenta del valore (%) per quell'azione
`ignore_disadvantage`	Lo svantaggio di elemento non si applica
`invert_advantage`	Vantaggi e svantaggi di elemento si invertono per entrambi
`copy_ability`	Copia l'abilità dell'avversario per il resto della battaglia